

AI Crypto Advisor

Moveo home assignment | Raz Gold

Links and access

Live app: <https://moveo-crypto-advisor-five.vercel.app>

GitHub repo: <https://github.com/RazGold98/moveo-crypto-advisor>

Backend API: <https://moveo-crypto-advisor-sadc.onrender.com>

Database access (read-only user for review):

```
mongodb+srv://moveo_reviewer:qqgCNOLKvtZDIGXX@firstproject.nzkolkq.mongodb.net/crypto-advisor
```

A quick note: the backend runs on Render's free tier, so it sleeps when idle. The first request after a while can take up to a minute to wake up.

What I built

A personalized crypto dashboard. When someone signs up they answer three quick questions, which coins they follow, what kind of investor they are, and what content they like, and the app uses those answers to build a dashboard around them. The dashboard has four parts: live coin prices, market news, an AI insight for the day, and a crypto meme. Each part has a thumbs up / thumbs down button, and those votes are saved so they can be used later to personalize the experience further.

Stack and structure

The frontend is React with Vite and TypeScript. The backend is NestJS, and the database is MongoDB on Atlas. This is the stack I work with day to day, so I could spend my time getting the structure and the details right instead of learning new tools. The backend is split into separate modules (auth, users, dashboard, votes), which keeps each feature self-contained and easy to follow.

The React app never talks to the outside crypto APIs or the database directly. It only calls my own backend and sends a login token with each request. The backend is the only place that holds the secret keys and connects to MongoDB and the external services. When the dashboard loads, the backend fetches all four sources at once and returns them together, already filtered to the user's preferences.

A few things I paid attention to

Passwords are never stored as plain text, they are hashed, and the backend always works out who the user is from the signed login token rather than from anything the browser claims. So a user can't act as someone else.

Every call to an outside service has a fallback. A couple of the free news APIs turned out to need a paid plan, so instead of letting that break the page, the app falls back to sensible defaults and keeps working. The AI insight and the meme work the same way.

I went with stable, proven versions of the tools instead of the newest experimental ones, after the bleeding-edge versions caused build failures. For something that has to run in production, reliable felt like the better call.

Where it runs

Database on MongoDB Atlas, backend on Render, frontend on Vercel, all on free tiers.

How I used AI

I used Claude (through Cowork) as a pair-programming partner and, honestly, as a tutor while building this. I didn't want it to just generate the whole app for me, because then I wouldn't really understand what I was handing in. So I worked through the project in order: it would explain a concept and suggest code, and I would write it, ask questions until it made sense, and move on. I can walk through any file in the project and explain why it's there.

I also made the calls on the parts that needed a decision. A few examples: choosing the stable versions of NestJS and Vite after the latest ones broke; adding the fallbacks when the news APIs changed their pricing so the app stayed within the free-API requirement; and tracking down two problems that only showed up in production, a native-dependency error that came from installing packages on a different operating system than the one running them, and CoinGecko blocking the server's IP, which I solved with a free demo key. It saved me time on the boilerplate and the explaining, but the decisions and the understanding are mine.